1) A university database must maintain accurate and consistent student information using appropriate database constraints.
   a) Construct a SQL CREATE TABLE statement for a Student table using appropriate PRIMARY KEY, NOT NULL, UNIQUE, and CHECK constraints.
2) A university maintains a database to manage information about its students, departments, courses, and instructors. Each student is uniquely identified by a Student ID and has a Name and Email. Every student belongs to exactly one department, while each department can have many students. Each department is uniquely identified by a Department ID and has a Department Name. The university offers several courses, where each course has a Course ID, Course Name, and Credit. Each course is offered by one department, while a department may offer many courses. The university also has instructors, where each instructor has an Instructor ID, Name, and Email. Each instructor can teach many courses, while each course is taught by one instructor. In addition, a student can enroll in many courses, and each course can have many students.
   a) Construct an appropriate ER diagram for the given system, showing the entities, attributes, relationships, and cardinalities.
3) A university database needs an instructor relation containing instructor ID, name, department, and salary. Each instructor must have a unique ID, and the department must reference the department relation.
   a) **Construct** an SQL create table statement satisfying the given requirements.
4)

ID	name	dept	salary
101	Smith	CS	60000
102	Kim	CS	80000
103	Wu	Finance	90000
104	Gold	Physics	70000
105	Darwin	Physics	NULL
106	Crick	Biology	75000

   a) **Write** an SQL query to display the name and dept of all instructors.
   b) **Write** an SQL query to display all distinct department names using DISTINCT.
   c) **Construct** an SQL query to display ID, name, and monthly salary by dividing salary by 12, and rename the calculated column as monthly_salary.
   d) **Construct** an SQL query to display the names of instructors belonging to the CS department with salary greater than 70000 using logical and comparison operators.

e) **Construct** an SQL query to display instructors who belong to either the CS or Physics department.

f) **Construct** an SQL query to display instructors whose names contain the substring i using the LIKE operator.

g) **Construct** an SQL query to display instructors whose names match a pattern of exactly four characters using _.

h) **Construct** an SQL query to display instructors in descending order of salary and then ascending order of name.

i) **Construct** an SQL query to display instructors whose salary is between 70000 and 90000 using BETWEEN.

j) C**onstruct** an SQL query using tuple comparison to identify the instructor whose (ID, dept) matches (102, 'CS').

k) **Construct** an SQL query using UNION to obtain instructors whose salary is greater than 80000 or whose department is Physics.

l) **Construct** an SQL query using INTERSECT to obtain instructors whose salary is greater than 70000 and whose department is Physics.

m) **Construct** an SQL query using EXCEPT to obtain instructors whose salary is greater than 70000 but whose department is not Physics.

n) **Construct** an SQL query using UNION ALL to retain duplicate results from two suitable queries.

o) **Construct** an SQL query to display instructors whose salary is NULL and another query to display instructors whose salary is not NULL.

p) **Calculate** the AVG, MIN, MAX, SUM, and COUNT values for the salary column.

q) **Construct** an SQL query using GROUP BY to display each department and its average salary.

r) **Construct** an SQL query using GROUP BY and HAVING to display departments whose average salary is greater than 70000.

s) **Construct** an SQL query using a nested subquery to display instructors whose salary is greater than the average salary of all instructors.

t) **Construct** an SQL query using a nested subquery to display the instructor(s) having the maximum salary.

u) **Construct** a Java JDBC code segment to open a database connection and create a Statement object using DriverManager.getConnection() and conn.createStatement().

v) **Demonstrate** the JDBC operations required to execute an SQL update using executeUpdate() and handle an SQLException.

w) **Construct** a Java JDBC code segment to execute a SELECT query using executeQuery() and retrieve the results using ResultSet and rset.next().

4) instructor(ID, name, dept, salary)

department(dept_name, building)

a) **Construct** the corresponding SQL CREATE TABLE statement for the given relation schema, using suitable data types and defining ID as the primary key.

b) **Construct** an SQL query to generate the Cartesian product of the instructor and department relations show the resultant table.